تقرير شامل — Zakai.io (مساحة عمل الأعمال)

التاريخ: 19 أكتوبر 2026 · الإصدار: 0.1.0 · المستودع: `MKory18/zaki-workspace`

---

١) نظرة عامة على المشروع

Zakai.io هو منصة مساحة عمل أعمال (Business Workspace) عربية بالكامل (RTL)، مبنية كـ Modular Monolith بواجهة واحدة وخلفية موحّدة. تخدم المؤسسات الصغيرة والمتوسطة التي تحتاج إدارة المشاريع والمهام والأقسام والعملاء والإنتاج اليومي في مكان واحد — بالعربية أصلًا، لا كترجمة لاحقة.

الوحدات المنجزة

الوحدة	الوصف
المؤسسات والمصادقة	تسجيل/دخول بـ bcrypt + جلسات JWT في httpOnly cookie، أدوار (مالك/مدير/موظف/زائر)، إدارة أعضاء وصلاحيات من صفحة `/members`
مساحات العمل والأقسام	مساحات عمل متعددة مع عزل صارم على مستوى الاستعلامات، أقسام داخل كل مساحة مع أعضاء
المشاريع واللوحات	لوحات Kanban بأعمدة مرتبة جزئيًا (fractional ordering)، سحب وإفلات متفائل، أعمدة "منتهي" تختم المهام تلقائيًا
المهام	أولويات، مواعيد، اعتماديات، تعليقات، نشاط موثّق، مفاتيح تسلسلية لكل مؤسسة (`ZD-42`)، تسميات (TaskActivity)
تذاكر IT	أول منتج داخل الأقسام: حالات (مفتوحة/قيد العمل/تم الحل/مغلقة) بمصفوفة انتقالات مفروضة، إسناد لأعضاء القسم، ترقيم `ZD-T{n}`
المبيعات CRM	عملاء وفرص بيع بمراحل (OpportunityStage) داخل الأقسام
الإنتاج اليومي	مهام متكررة (روتين) تُولّد مهامًا يومية وتُوزَّع على الموظفين
الأهداف	أهداف أسبوعية وشهرية بشريط تقدم قابل للتحديث الجزئي
الملفات والأفكار والدردشة	رفع ملفات (تخزين محلي)، بنك أفكار، دردشة ورسائل
الإشعارات	جرس حي مع تحديد كمقروء لكل إشعار وحذف، مرتبط بكل الأحداث (إسناد مهمة، تعليق، إضافة عضو، تغيير دور، إسناد تذكرة...)
الأمن	فحص صحة `env.ts` fail-fast، عزل Zod تحقق على كل مسار، صلاحيات على الخادم حصريًا، نفسها `where` المؤسسات في

اللغة والاتجاه

عربية حصرًا (أُزيلت الفرنسية والإنجليزية بطلب المستخدم). القاموس `src/lib/i18n/ar.json`، اتجاه RTL تلقائي عبر CSS منطقي (`ms-` / `me-`)، خط IBM Plex Sans Arabic.

---

٢) البنية والتقنيات (Architecture Report)

المكدس التقني

الطبقة	التقنية
الواجهة	Next.js 14 App Router (React Server Components) + Tailwind 3 + TanStack Query
الخادم	Next.js route handlers + طبقة خدمات `src/server/services/*`
قاعدة البيانات	PostgreSQL عبر Prisma ORM، مهاجرات إضافية فقط (additive-only)
التحقق	Zod على كل جسم طلب
الأمن	bcrypt(12)، HS256 session cookie (httpOnly, SameSite=Lax, Secure في الإنتاج)
الخلفية	worker مستقل (`worker/index.ts`) — BullMQ 5 مخطط للمرحلة+
الاختبارات	Vitest — 76/76 تمر (12 ملف)

الطبقات

```
UI (مكونات خادمية تقرأ Prisma مباشرة للعرض)
 → مكونات عميل تُغذّى عبر TanStack Query مع /api/*
   → مسارات API (تحقق Zod + صلاحيات requireX)
     → خدمات (src/server/services/*) — قواعد العمل والمعاملات
       → Prisma — التخزين
```

القواعد الذهبية:

- **مصدر حقيقة واحد لكل مفهوم**: خدمة المهام هي المسار الوحيد لتعديل المهام؛ الواجهة وتيليجرام والأتمتة كلها تستدعيها.
- **العرض يقرأ مباشرة، التعديل يمر دائمًا عبر API + الخدمات** — لا استدعاءات Prisma من الواجهة للكتابة.
- **الصلاحيات على الخادم لكل طلب**: `requireUser` ، `requireOrgMember(orgId)` ، `requireWorkspace(wsId, roles)` كل خدمة تحل سلسلة الملكية داخل معاملتها الخاصة.
- **التصريح الصاخب**: كل مسار API ملفوف بـ `withApi` يربط الأخطاء المُنمَّطة برموز HTTP؛ الأخطاء المجهولة تُسجَّل بلا stack traces) ومنظّم وتُعاد 500 مبهمة JSON).

نموذج البيانات الأساسي

```
Organization ─< OrganizationMember >─ User
Workspace ─< WorkspaceMember
Workspace ─< Team ─< Project
Project ─< Board ─< BoardColumn ─< Task
Task ─< TaskComment / TaskActivity / TaskLabel >─ Label / TaskDependency
Notification, AuditLog, Customer, Opportunity, Ticket, Routine, Goal
```

- **مفاتيح المهام**: عدّاد `TaskCounter` لكل مؤسسة، يُزاد داخل معاملة الإنشاء؛ `@@unique([orgId, number])` يمنع السباقات.
- **الترتيب الجزئي**: النقل/الإنشاء يكتب صفًّا واحدًا بـ `position = midpoint(prev, next)` — لا إعادة ترقيم جماعية.
- **دلالة "منتهي"**: عمود اللوحة يحمل علم `done` ؛ النقل إليه يختم `completedAt` — سير العمل بيانات لا كود.

التزامن والسلامة

- الكتابات متعددة السجلات داخل `$transaction` (إنشاء مشروع/لوحة/مهمة، نقل).
- السحب والإفلات متفائل في الواجهة؛ عند الفشل يستعيد الحالة السابقة — النقل المرفوض لا يبدو ناجحًا أبدًا.
- عزل حدود المساحة على مستوى الاستعلام (org + workspace joins في `where`).
- الزائر (GUEST) للقراءة فقط؛ الكتابة تتطلب ADMIN/MEMBER.

نقاط الامتداد المجهزة

- `worker/index.ts` — مكان واحد لكل المهام الخلفية (تيليجرام، أتمتة، مهام متكررة، ملخصات AI).
- `src/server/services/tasks.ts` — كل تعديلات المهام؛ أي سطح جديد يستدعي الخدمات، لا Prisma مباشر.
- `env.ts` — عقد بيئة مُنمَّط؛ التكاملات الجديدة (Telegram/AI) تمتد فيه وتبقى معطّلة عند فراغ المفاتيح.

---

٣) كيفية إنشاء وتشغيل المشروع

المتطلبات

- Node.js 20+ · PostgreSQL 14+ (أو Docker)

الإعداد المحلي

```
# 1) استنساخ وتثبيت
git clone https://github.com/MKory18/zaki-workspace.git
cd zaki-workspace
npm install

# 2) البيئة
cp .env.example .env
#   DATABASE_URL="postgresql://zaki:<pass>@localhost:5432/zaki_workspace"
#   AUTH_SECRET=<سلسلة عشوائية طويلة>

# 3) قاعدة البيانات (Docker) أو خدمة Postgres محلية
docker compose up -d db
npm run db:migrate        # يطبق المهاجرات

# 4) التطوير
npm run dev               # http://localhost:3000
```

البوابة قبل الإصدار

```
npm run typecheck   # tsc --noEmit — يجب أن يكون نظيفًا
npm test            # vitest — 76/76
npm run build       # prisma generate && next build — بوابة الإنتاج
```

النشر الإنتاجي (Docker)

```
cp .env.example .env         # عيّن AUTH_SECRET و POSTGRES_PASSWORD
docker compose up -d migrate # المهاجرات مرة واحدة
docker compose up -d app worker
```

- قاعدة البيانات محفوظة في `db_data`، الملفات في وحدة `uploads`.
- النسخ الاحتياطي: `docker compose exec db pg_dump -U zaki zaki_workspace > backup.sql`
- الاستعادة: `docker compose exec -T db psql -U zaki zaki_workspace < backup.sql`

---

٤) الربط بالنشر على Coolify

Coolify يدعم Next.js مباشرة عبر Dockerfile الموجود في المستودع (أو Build Pack "Node.js"). الخطوات:

الطريقة A — من GitHub (موصى بها)

1. **المستودع جاهز**: `MKory18/zaki-workspace` — كل الالتزامات مرفوعة، آخرها وحدة CRM.
2. **في Coolify**: مشروع جديد ← **New Resource** ← **Docker Compose** أو **From Git Repository** ← اختر `MKory18/zaki-workspace` والفرع `main`.
3. **Build & Deploy**:
   - إذا Coolify يستخدم Dockerfile — بناء المرفق: لا شيء تُعدّله — `next build` داخل الصورة.
   - إذا Build Pack: أمر البناء `npm run build` (يشمل `prisma generate` تلقائيًا) وأمر التشغيل `npm start`.
4. **متغيرات البيئة** (Environment Variables في Coolify) — المطلوبة إلزاميًا:

المتغير	القيمة
`DATABASE_URL`	رابط Postgres مُدار (Neon/Supabase/Railway) مع `?sslmode=require` — أو خدمة Postgres داخلية في Coolify
`AUTH_SECRET`	سلسلة عشوائية طويلة (`openssl rand -base64 48`)
`APP_URL`	دومين النشر مثال (`https://zakai.<دومينك>`)

   الاختيارية (تبقى معطّلة عند فراغها): `TELEGRAM_BOT_TOKEN`، `GOOGLE_CLIENT_ID/SECRET`، `REDIS_URL`، مفاتيح LLM.

5. **قاعدة البيانات** — خياران:
   - **خدمة Postgres داخل Coolify** (أسهل كل شيء ذاتي الاحتياط): أضف من نوع Resource في PostgreSQL بالخدمة الداخلية `DATABASE_URL` نفس المشروع، واربط.
   - **مُدار خارجي Postgres** (Neon مجاني): الصق رابطه في `DATABASE_URL`.
6. **المهاجرات** — مرة واحدة بعد أول نشر (منفذ الأوامر Coolify من طرفك أو من طرف):

```
DATABASE_URL="<الرابط>" npx prisma migrate deploy
```

   المهاجرات **إضافية فقط** — إعادة التشغيل آمنة دائمًا.

7. **HTTPS والدومين والـ**: Coolify يدير الوكيل العكسي (Traefik) والشهادات تلقائيًا — اربط الدومين في إعدادات التطبيق (Domains) مقابل المنفذ 3000.
8. **التحقق بعد النشر**: افتح `https://<دومينك>/api/health` — يجب أن يعيد حالة المكونات دون أي أسرار أو تفاصيل.

ملاحظات إنتاجية مهمة

- **رفع الملفات**: يُكتب على القرص المحلي (`.data/uploads`) في نشر Coolify بأحجام دائمة (Volumes) يبقى؛ لو استضفت على Vercel فسيلزم ربطه بـ S3/Vercel Blob (نقطة الكتابة الوحيدة في `src/server/services/files.ts`) لاحقًا.
- **Worker/Redis**: متوفرتان في docker-compose. اختياريان للنشر الذاتي فقط؛ خدما app+worker.

- **النسخ الاحتياطي على Coolify**: جدّد نسخًا دوريًا لخدمة Postgres (أو أضف مهمة مجدولة `pg_dump`).

ملخص الجاهزية

الحالة	البند
✅	المهاجرات (10) إضافية وآمنة
✅	فحص fail-fast للمتغيرات
✅	docker-compose كامل (app + worker + db + migrate)
✅	اختبارات 76/76 + بناء أخضر
✅	`.env.example` موثق بالكامل
✅ (`main -> main`)	رفع أحدث كود إلى GitHub

التقرير قابل للتحويل إلى PDF: افتح الملف في VS Code (امتداد "Markdown PDF") أو استخدم `npx md-to-pdf docs/PROJECT-FULL-REPORT.md`.